ANITS

INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1.	8/7/26	Introduction to HTML and CSS	CO1	10	
2.	15/7/26	static webpage Using HTML and CSS	CO1	10	
3.	15/7/26	HTML Codes Using audio & video	CO1	10	
4.	22/7/26	Program Using JavaScript	CO1	10	
5.	29/7/26	Dynamic webpage Using Javascript	CO2	10	
6.	5/8/26	Implement modules in Node Js	CO2	10	
7.	12/8/26	Develop miniapplication Using Express and NodeJs	CO2	10	
8.	9/9/26	functional Component & class Component Using react.	CO2	10	

INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
9.	23/9	Implement http method with JSON server	Co_2	10	
10.	30/9	Develop CRUD operation using mongoDB	Co_3	10	
				10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	**Semester:**	3rd 1st
Name of the Student:	B.Chandra Moulika	**Roll Number:**	A24126552131
Name of the Lab:	FS Lab	**Timings:**	9:40AM to 12:10PM

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	1	4	4	5	2	15		
8/7	2	4	4	5	1	14		
15/7	3	4	4	5	2	15		
22/7	4	4	4	5	1	14		
29/7	5	4	4	5	2	15		
5/8	6	4	4	5	1	14		
12/8	7	4	4	5	2	15		
19/8	08	4	4	5	1	14		
2/9	09	4	4	5	2	15		
7/9	10	4	4	5	2	15		good.

Name of the Faculty: Ch. Swathi

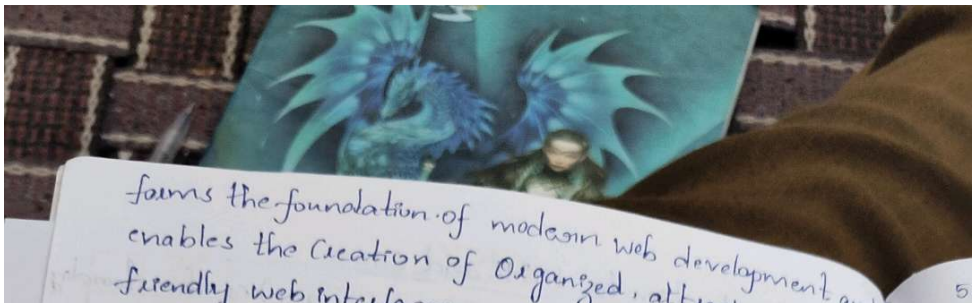
Faculty Signature: Swathi

* Introduction to HTML and CSS:

HTML (HyperText Markup Language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interepted by web browsers to display the content in a structured and meaningful manner. HTML provides structure and semantic organization of a webpage.

CSS (Cascading Style Sheets) is a style sheet language used to control the presentation & visual appearance of HTML Documents. It is used to define Properties such as colors, fonts, background, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sized and devices. CSS can be applied using inline, internal or external.

HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides styling. The combination of both



forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

* Difference between MERN stack and FULL stack:

MERN stack is a java script based technology stack used for developing modern web applications. MERN stack stands for MongoDB, Express.js, React.js, Node.js, It used for database management, express.js handles the backend framework and API's, React.js is used to develop frontend User Interface. and node.js provides Server side runtime environment.

→ FULL-stack Development refers to the development of both frontend and backend of a web application along with database integration. A Full stack developer works with technologies.

* HTML Elements:

1. HTML Tags- Define the structure and elements of a webpage.
2. Headings Used to display titles and headings Using <h1> to <h6>
3. Paragraphs Used to display text Using <p> tag.
4. links Used to navigate between webpages Using <a> tags.

5. Images - Used to display images using <image> tag.
6. Lists - Used to create ordered & Unordered lists using <ol>, <ul>, <li>.
7. Tables - Used to organize data in rows and columns.
8. Forms - Used to collect user input using elements such as textboxes, buttons and check boxes.
9. Div and Span - Used to group and organize webpage content.
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provides meaningful webpage structure.

CSS Components:

1. Selectors - Select HTML elements to apply styles.
2. Properties - Define the visual characteristics of elements.
3. Values - Specify the setting of a CSS property.
4. Colors - Define text, background, borders, colors.
5. fonts - Control font family, size, style and weight.
6. Background - Set background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - Controls space outside an element.
9. Padding - Controls the space inside an element.
10. Layout - Controls arrangement of elements using

flexbox, grid and positioning.

* HTML Vs HTML5:

HTML	HTML5
· Older version of HTML.	· Latest major version of HTML.
· Limited multimedia support.	· Supports audio and video.
· Uses external plugins for multimedia.	· No external plugs needed.
· Has fewer semantic tags.	· Provides semantic tags.
· No <Canvas> element limited webstorage.	· Support <canvas> Supports local storage.

* CSS Vs CSS3:

CSS	CSS 3
· Earlier version of CSS.	· Advanced version of CSS
· Basic styling features	· Provides advanced styling features.
· Has limited animations.	· Supports animations and transitions.
· No flexible layout systems.	· Supports flexbox and Grid.

① Design a static webpage using HTML and CSS.

→ pomodoro Timer - Using HTML, CSS.

```
<!doctype html>
<html>
 <head>
    <title>pomodoro Timer</title>
    <style>
      body {
        font-family: Arial;
        background: #f5f5f5;
        text-align: center;
        Padding: 50px;
      }
     timer {
       width: 300px;
       margin: auto;
       Padding: 30px;
       background: white;
       border-radius: 15px;
       box-shadow: 00 10px grey;
      }
     h1 {
        color: #e74c3c;
      }
```

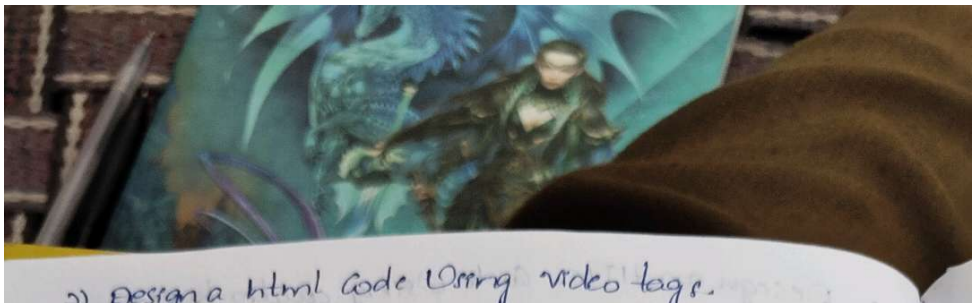
```
time {
    font-size: 50px;
    margin: 20px;
}
button {
    padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius: 5px;
    background: #e74c3c;
    color: white;
}
</style>
</head>
<body>
    <div class="timer">
    <h1> pomodoro Timer </h1>
    <div class="time"> 25:00 </div>
    <button> start </button>
    <button> Reset </button>
</div>
</body>
</html>
```

1. Design an HTML code using audio tags:

Code:

```
<!Doctype html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> my favourite song </h2>
    <audio controls>
        <source src = "Song.mp3" type =
                "audio/mpeg">
        Your browser does not support audio tag.
    </audio>
</body>
</html>
```

O/P: Displays an audio player with play, pause, and volume controls.



2) Design a html code using video tags.

Code:

```
<!Doctype html>
<html>
<head>
   <title> video example </title>
</head>
 <body>
    <h2> my video </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4">
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and full screen controls.

3. Write a program using Javascript (Arrays, functions).

Code:

```
let numbers = [ 12,45,8,23,56,19,34,67,90,
Function display(arr){                    ];
    Console.log("Array elements:");
    for( let i=0;i< arr.length;i++){
        Console.log(arr[i]);
    }
}
Function FindSum(arr){
    let Sum=0;
    for(let i=0;i< arr.length;i++)
        Sum+=arr[i];
    }
    return sum;
}
```

Experiment-04

4) Develop Dynamic Webpage Using Javascript.

Program:

```
<!DOCTYPE Html>
<html>
<head>
  <title> Dynamic webpage </title>
  <style>
    body{
      text-align: center;
      font-family: Arial;
      Margin-top: 100px;
    }
   button {
      Padding: 10px 20px;
      font-size: 16px;
      Cursor: pointer;
    }
</style>
</head>
<body>
  <h1 id="heading> welcome to M
  </h1>
```

```
<p id = "message"> click the button to change this
                                webpage </p>
<button onclick, "change page( )" > click me
                                </button>
<script>
    function change page( ){
        document.getElementById ("heading").
            innerHTML = "welcome to javascript!";
        document.getElement By Id ('message").innerHTML
            = "The webpage has been changed
                                Successfully";
        document.body.stylebackgroundcolour = "Lightblue";
        }
</script>
</body>
</html>
```

output:

Experiment-05

5) Implement modules in Node.JS:

In Node.js, you can implement modules using two distinct systems: CommonJS (CJS), which is traditional default, and ES modules (ESM), the modern javascript standard.

1. The CommonJS method (default):

CommonJS uses module.exports to share code & require() to import it. This system loads modules synchronously.

a. Create the module (math.js):

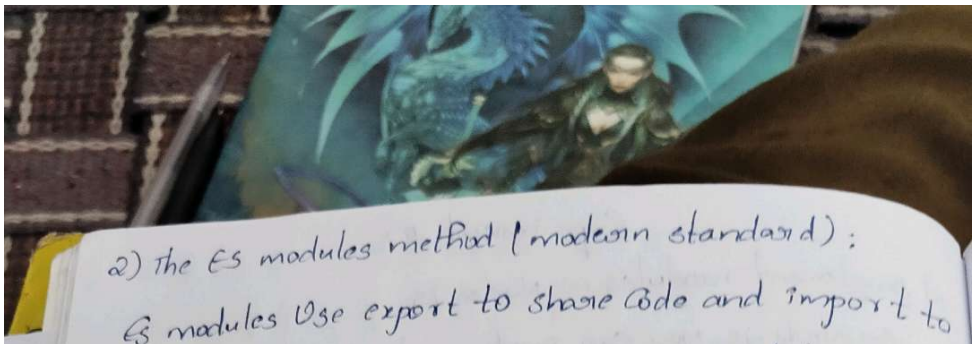
```
Const add = (a,b) => a+b;
Const Subtract = (a,b) => a-b;
module.exports = {
        add,
        Subtract
};
```

b. Use the module (app.js):

```
Const math = require('./math');
Console.log(math.add(10,5));
Console.log(math.Subtract(10,5))
```

output:

```
15
5
```



2) the ES modules method (modern standard):

ES modules use export to share code and import to use it. To make Node.js recognize ES modules, you must either use the .mjs file extension or add a "type": "module" to your package.json file.

a. Create the module (calculator.js)

```
export const multiply = (a, b) => a * b
export const divide = (a, b) => a / b
const CalculatorName = "Supercalc";
export default CalculatorName;
```

b. Use the module (main.js)

```
import CalculatorName, {multiply, divide}
                    from './calculator.js';
Console.log(CalculatorName);
Console.log(multiply(4, 2));
Console.log(divide(4, 2));
```

outputs:

```
Supercalc
8
2
```

3. Using Built-in Core modules:

Node.js also comes equipped with dozens of built-in core modules (like fs for file systems or path for file routing) that don't need installation. We will load them by module names:

```
// Common JS Style
Const fs = require ('fs');
// ES module style
import fs from 'fs';
```

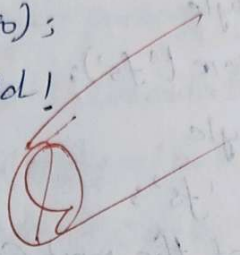
Here are some of the most commonly used ones;

1. fs - file system operations.
2. http - HTTPServer and client.
3. path - FilePath utilities.
4. OS - operating System utilities.
5. events - Event handling
6. util - utility functions.
7. stream - stream handling
8. Crypto - Cryptographic functions.
9. Url - URL parsing.
10. querystring - URL query string handling

Example: Simple HTTP Server.

```
http.createServer( (req, res) => { res.write
        (200, {'Content-Type': 'text/html
res.end ('Hello World!');
}), Listen (8080);
```

output: Hello World!



Experiment-06

6) Develop mini application Using express and NodeJs.

.Simple mini application Using Node JS terpress JS.

Student Management System:

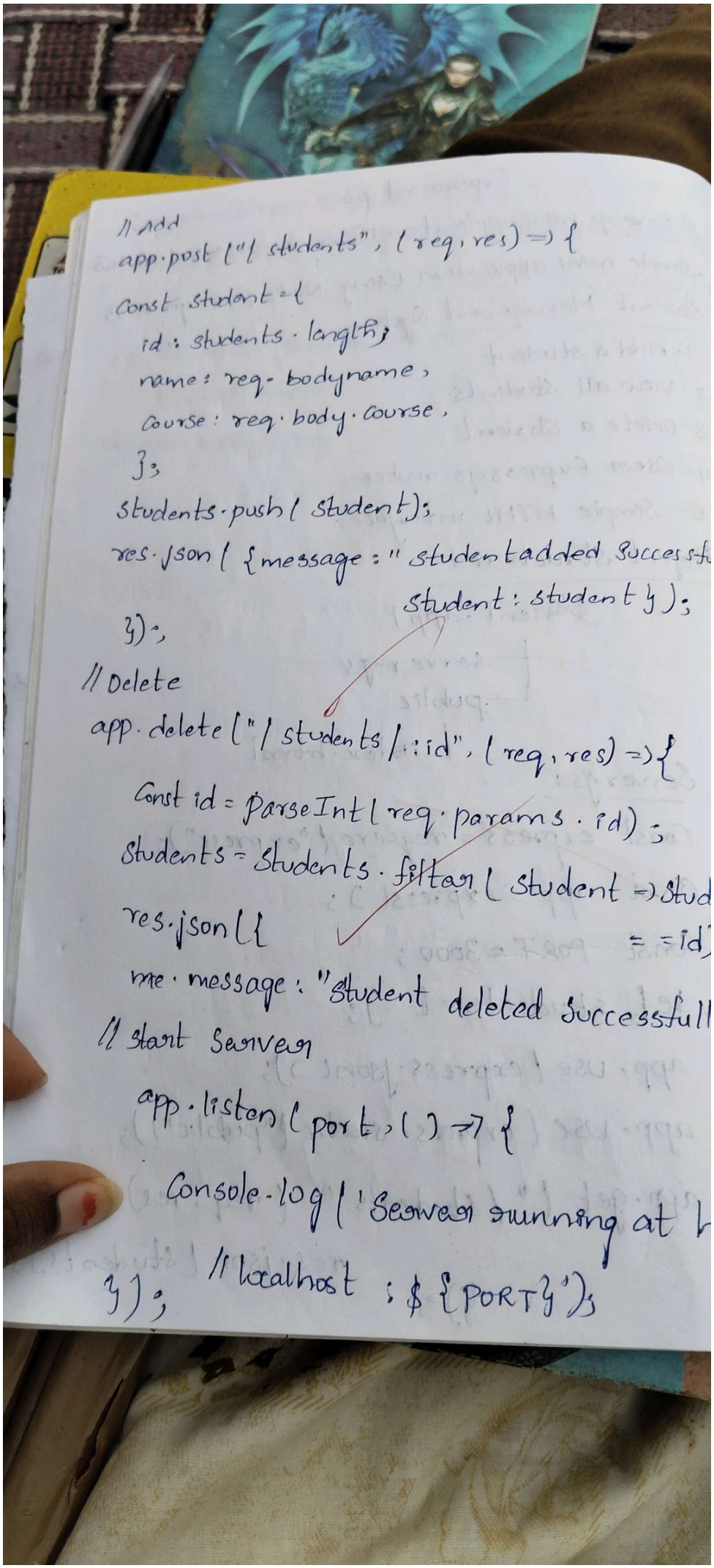
1. Add a student
2. View all students.
3. Delete a student
4. Use Express.js routes
5. Simple HTML interface.

Project Structure:

```
Student-app/
├── server.js
└── public
    └── index.html
```

Server.js:

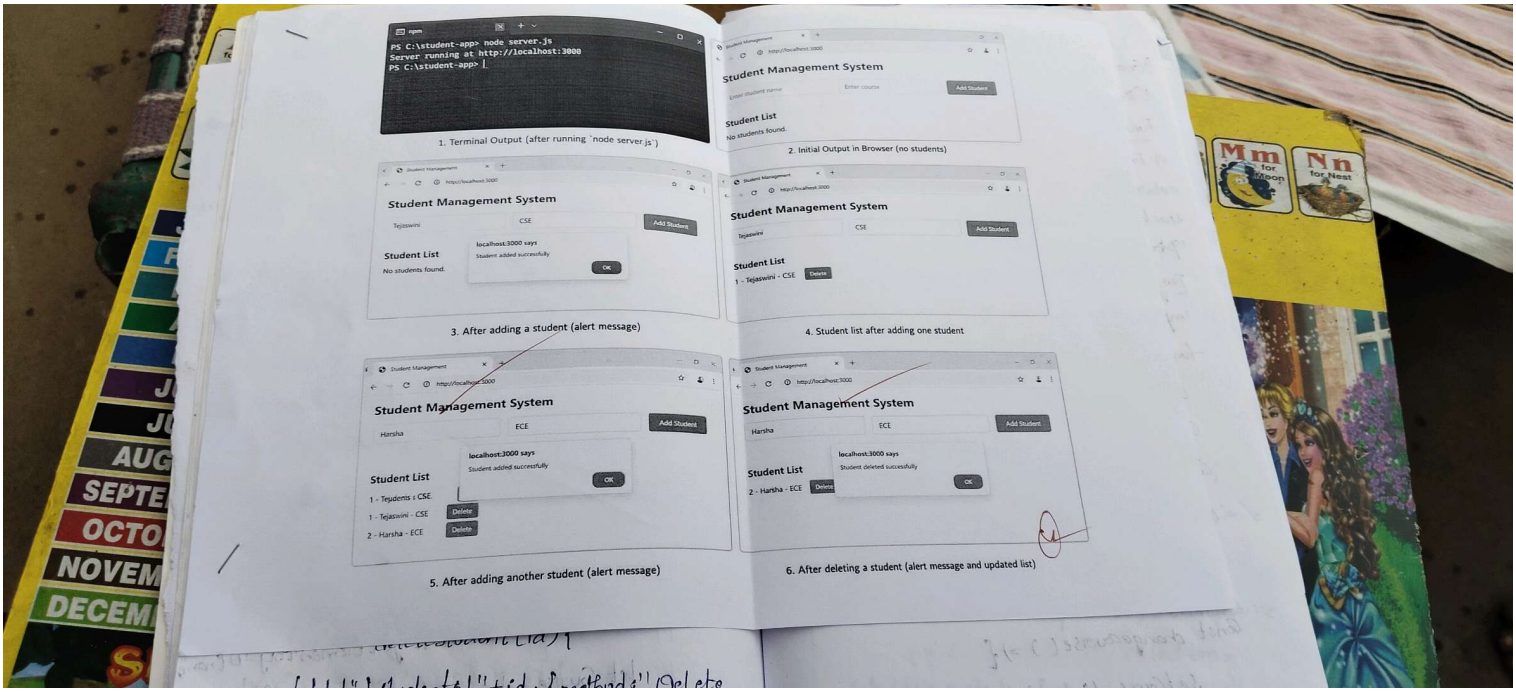
```
Const express = require("express");
Const app = express();
Const PORT = 3000;
let students = [ ];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
                    res.json(student
});
```

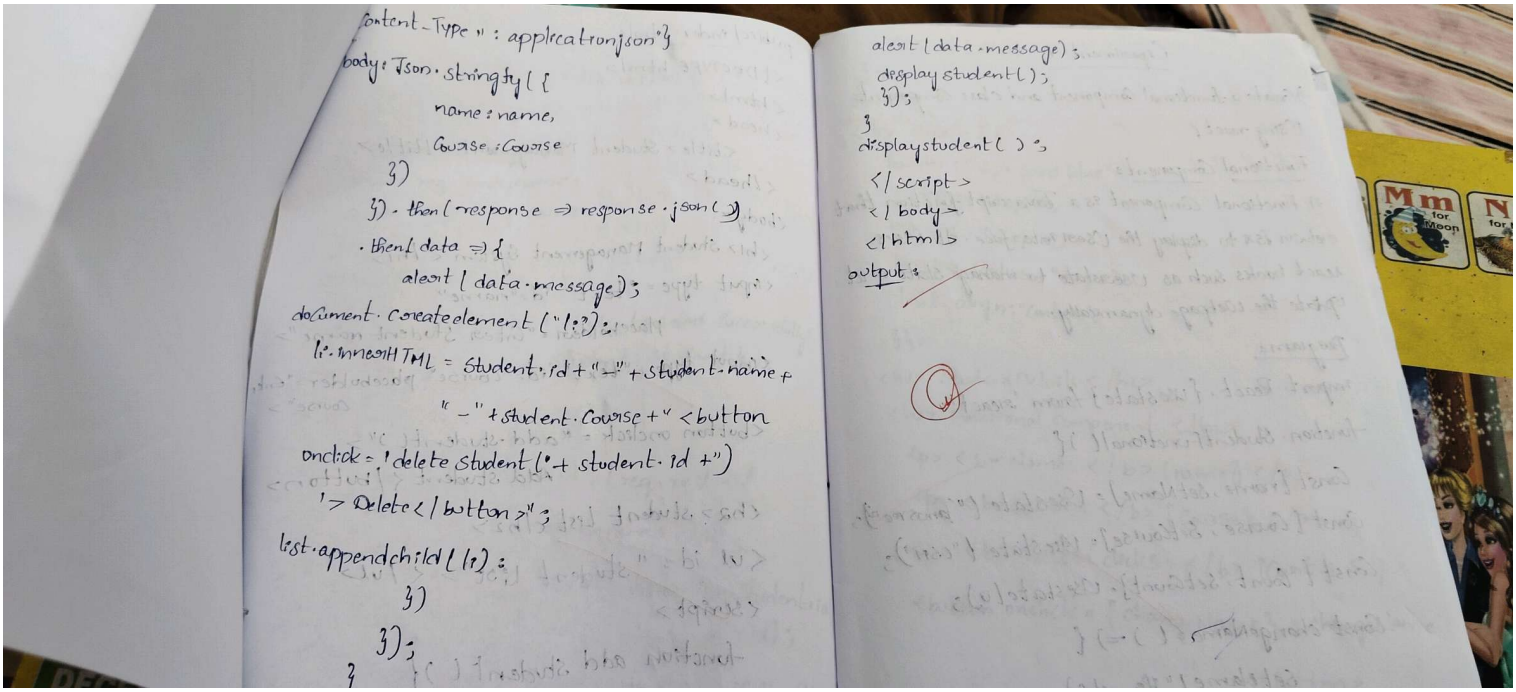


```
// Add
app.post("/students", (req, res) => {
const student = {
  id: students.length,
  name: req.body.name,
  course: req.body.course,
  };
students.push(student);
res.json({ message: "student added Success-fu
                                student: student });
});
// Delete
app.delete("/students/:id", (req, res) => {
  const id = parseInt(req.params.id);
  students = students.filter(student => stud
                                    === id)
  res.json({
  me.message: "student deleted successfull
// start Seerver
  app.listen(port, () => {
    Console.log('Seerver running at h
// localhost:${PORT}`);
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
  <h1> Student Management System </h1>
  <input type = "text" id="name"
          Placeholder = "Enter Student name">
  <input type = "text" id= "Course" placeholder = "Enter
                                                  Course">
  <button onclick = "add.student()">
                    Add student </button>
  <h2> student List </h2>
  <ul id : "student List"> </ul>
  <script>
    function add student() {
        let name = document.getElementById("n
                                          .value;
        let Course = document.getElementById("Coo
                                          value;
      fetch("/students", { method : "POST",
                           headers: {
```

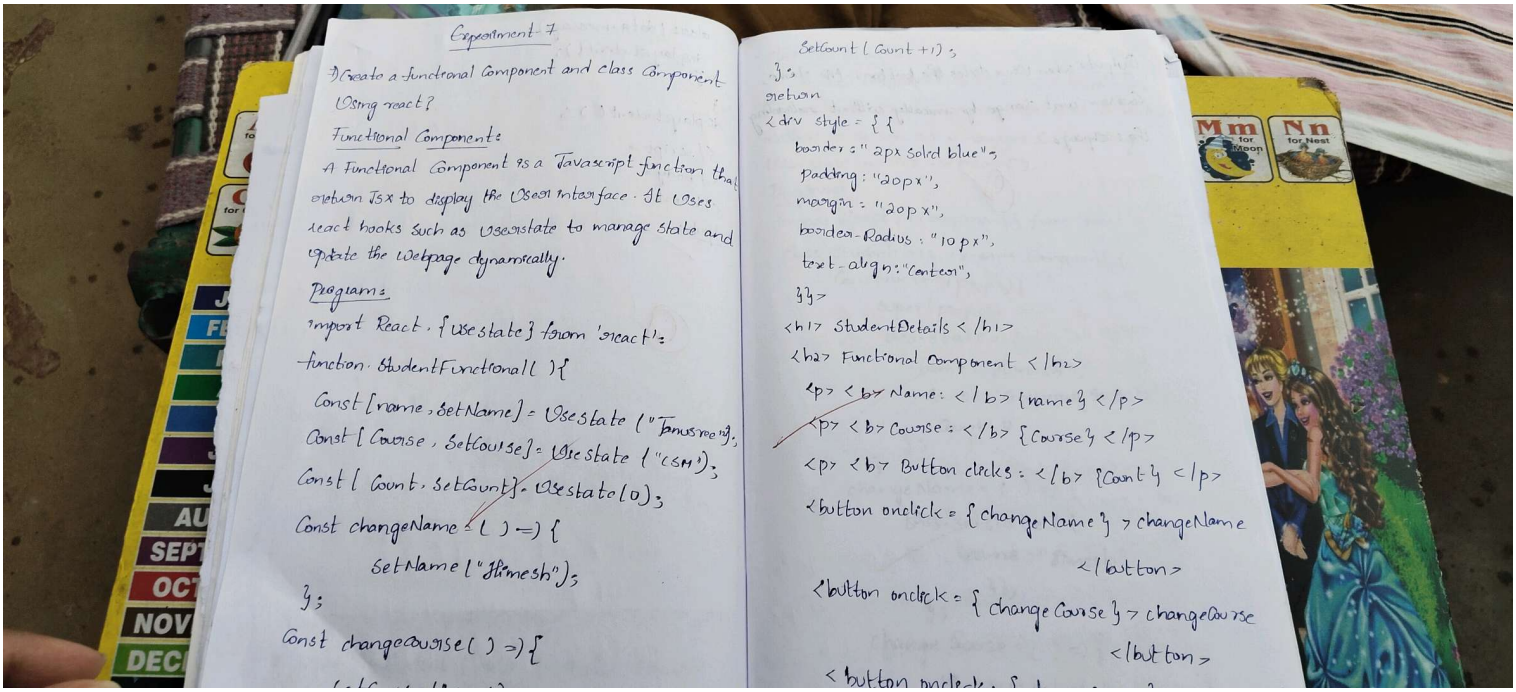

PS C:\student-app> node server.js
Server running at http://localhost:3000
PS C:\student-app>
1. Terminal Output (after running `node server.js`)
Student Management System
Student List
No students found.
2. Initial Output in Browser (no students)
Student Management System
Tejaswini
CSE
Add Student
localhost:3000 says
Student added successfully
OK
Student List
No students found.
3. After adding a student (alert message)
Student Management System
Tejaswini
CSE
Add Student
Student List
1 - Tejaswini - CSE
Delete
4. Student list after adding one student
Student Management System
Harsha
ECE
Add Student
localhost:3000 says
Student added successfully
OK
Student List
1 - Tejaswini - CSE
Delete
2 - Harsha - ECE
Delete
5. After adding another student (alert message)
Student Management System
Harsha
ECE
Add Student
localhost:3000 says
Student deleted successfully
OK
Student List
2 - Harsha - ECE
Delete
6. After deleting a student (alert message and updated list)



```
'Content-Type': 'application/json'},
body: JSON.stringify({
        name: name,
        Course: Course
    })
}).then(response => response.json())
.then(data => {
    alert(data.message);
document.createElement("li");
li.innerHTML = Student.id + "-" + Student.name +
        "-" + Student.Course + "<button
onclick='deleteStudent(" + student.id + ")
'>Delete</button>";
list.appendChild(li);
    });
});
```

```
alert(data.message);
displayStudent();
});
}
displayStudent();
</script>
</body>
</html>
```

output:



Experiment-7

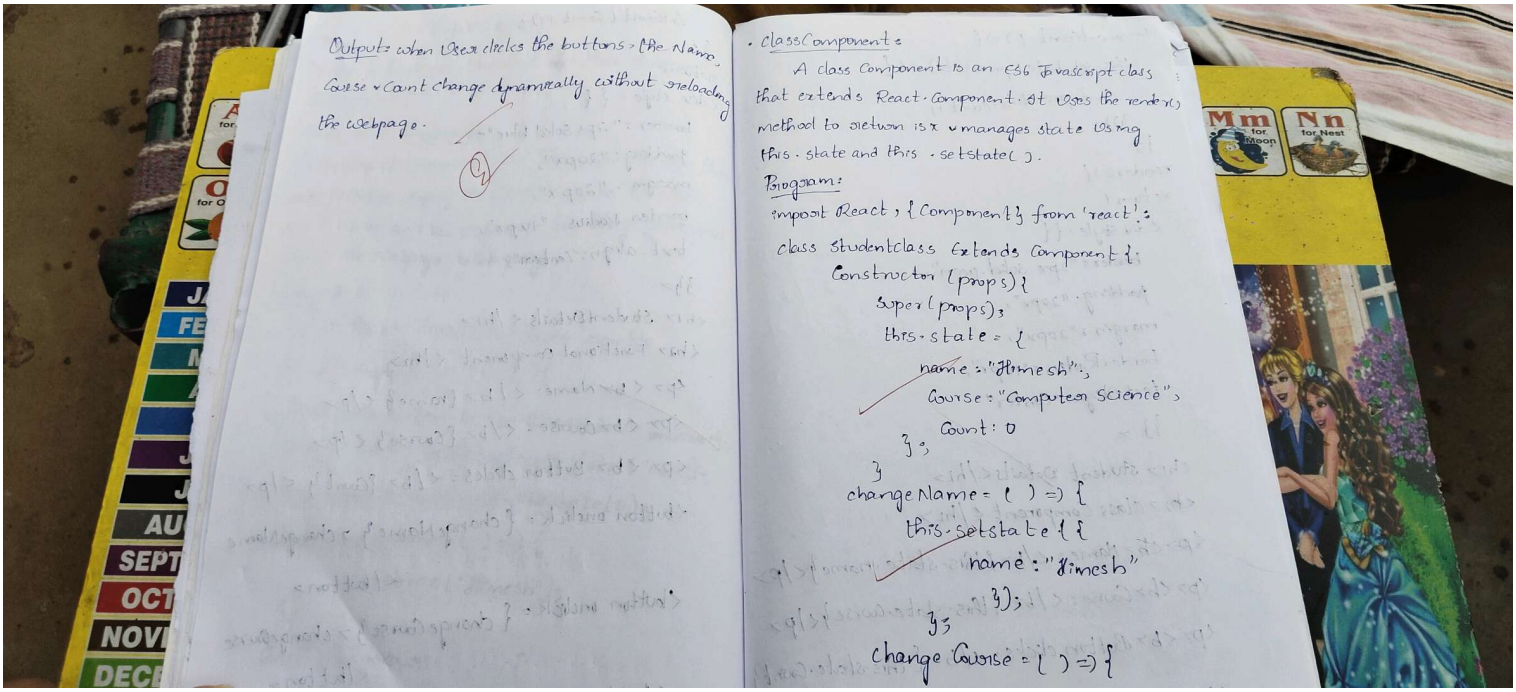
1) Create a functional component and class component using react?

Functional Component:

A Functional Component is a Javascript function that returns JSX to display the User interface. It uses react hooks such as Usestate to manage state and update the Webpage dynamically.

Program:

```
import React, {useState} from 'react';
function StudentFunctional(){
  Const [name, setName] = Usestate ("Tanusree");
  Const [Course, setCourse] = Usestate ("CSM");
  Const [Count, setCount] = Usestate(0);
  Const changeName = () => {
        setName("Hitesh");
  };
  Const changeCourse() => {
        setCount(Count+1);
  };
  return
  <div style={{
    border: "2px solid blue",
    padding: "20px",
    margin: "20px",
    border-Radius: "10px",
    text-align: "center",
  }}>
  <h1> StudentDetails </h1>
  <h2> Functional Component </h2>
    <p> <b> Name: </b> {name} </p>
    <p> <b> Course: </b> {Course} </p>
    <p> <b> Button clicks: </b> {Count} </p>
    <button onclick={changeName}> changeName
                                  </button>
    <button onclick={changeCourse}> changeCourse
                                    </button>
    <button onclick={
```



Output: when User clicks the buttons, the Name, Course & Count change dynamically without reloading the webpage.

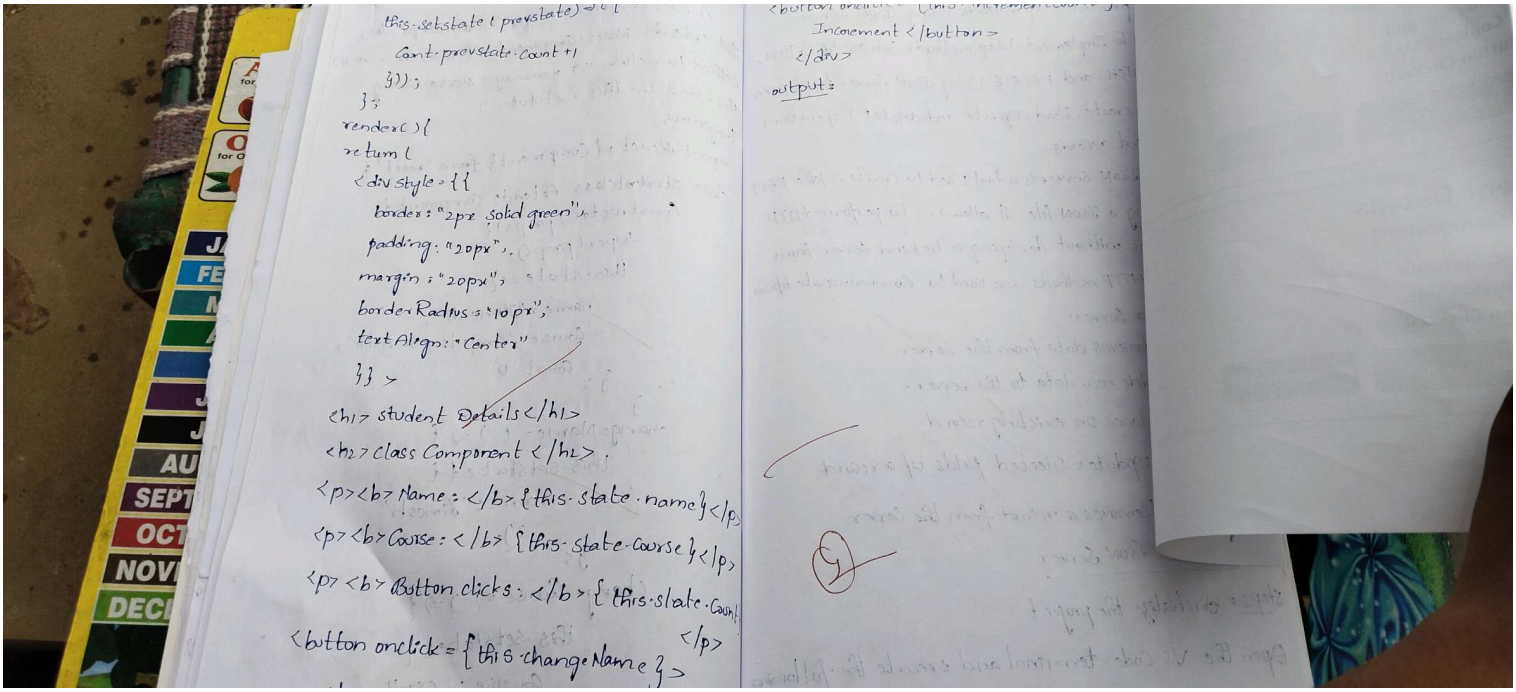
Q1

- classComponent:

A class Component is an ES6 Javascript class that extends React.Component. It uses the render() method to return JSX & manages state using this.state and this.setState().

Program:

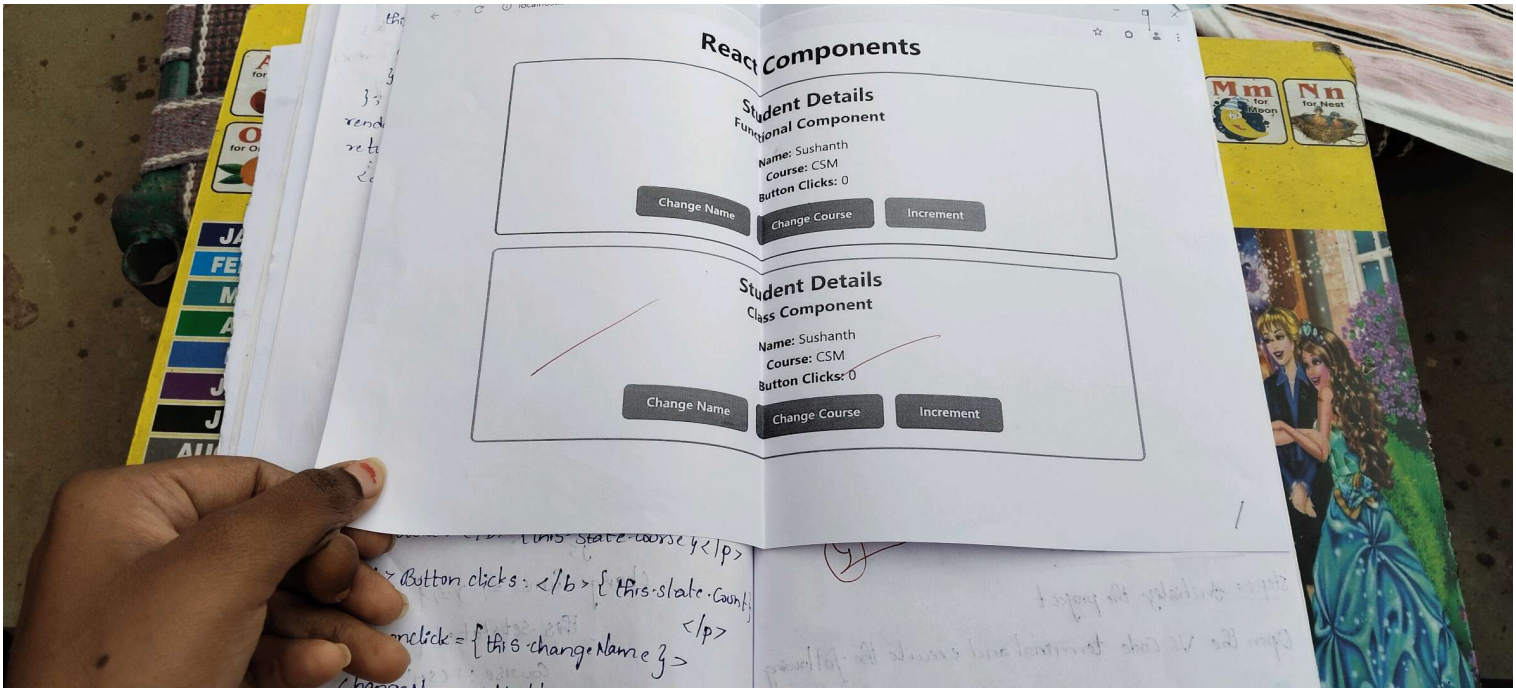
```
import React, {Component} from 'react';
class StudentClass Extends Component {
    Constructor (props) {
        Super (props);
        this.state = {
            name: "Himesh",
            Course: "Computer Science",
            Count: 0
        };
    }
    changeName = ( ) => {
        this.setstate {{
            name: "Yimesh"
        });
    }
    Change Course = ( ) => {
```

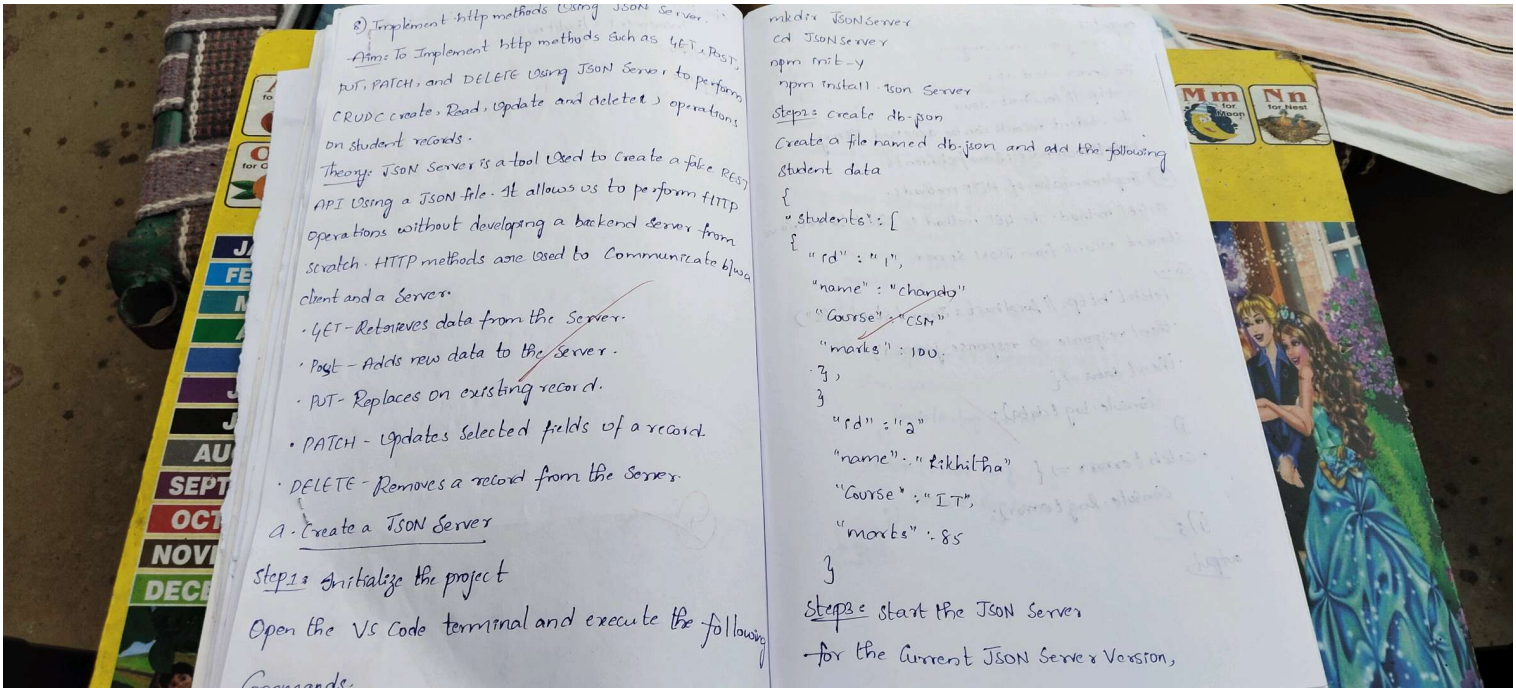


```
this.setState(prevState => {
  Cont: prevState.Count + 1
});
};
render() {
return (
<div style={{
  border: "2px solid green",
  padding: "20px",
  margin: "20px",
  borderRadius: "10px",
  textAlign: "Center"
}}>
<h1>student Details</h1>
<h2>Class Component</h2>
<p><b>Name:</b>{this.state.name}</p>
<p><b>Course:</b>{this.state.course}</p>
<p><b>Button clicks:</b>{this.state.Count}</p>
<button onClick={this.changeName}>
```

```
Incomement</button>
</div>
```

output:


React Components
Student Details
Functional Component
Name: Sushanth
Course: CSM
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Sushanth
Course: CSM
Button Clicks: 0
Change Name
Change Course
Increment



8) Implement http methods using JSON Server.

Aim: To Implement http methods such as GET, POST, PUT, PATCH, and DELETE using JSON Server, to perform CRUDC create, Read, Update and delete) operations on student records.

Theory: JSON Server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch. HTTP methods are used to communicate b/wa client and a server.

- GET - Retrieves data from the server.
- Post - Adds new data to the server.
- PUT - Replaces on existing record.
- PATCH - Updates selected fields of a record.
- DELETE - Removes a record from the server.

A. Create a JSON Server

Step1: Initialize the project

Open the Vs Code terminal and execute the following commands.

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server
```

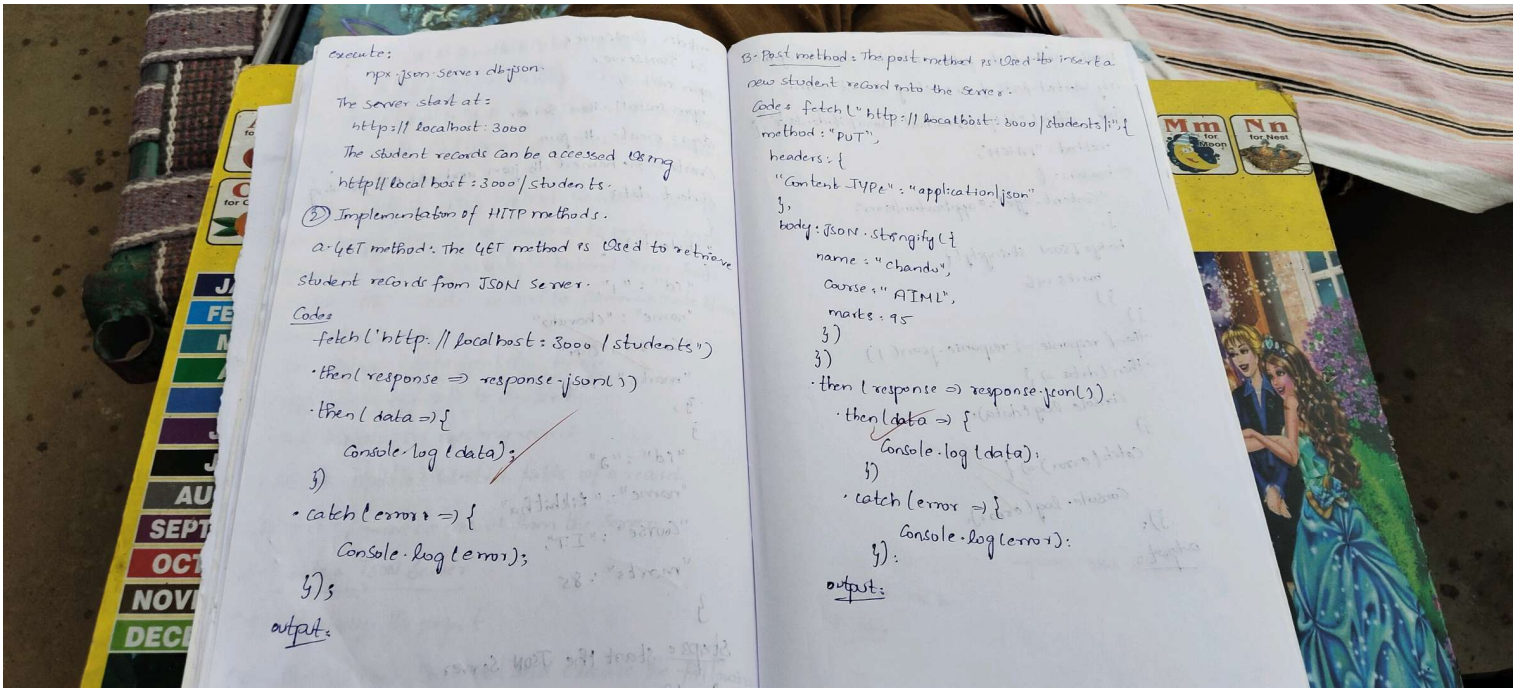
Step2: create db.json

Create a file named db.json and add the following student data

```
{
 "students": [
  {
   "id": "1",
   "name": "chandu"
   "Course": "CSM"
   "marks": 100.
  },
  {
   "id": "2"
   "name": "Likhitha"
   "Course": "IT",
   "marks": 85
  }
```

Step3: start the JSON Server

for the current JSON Server Version,



Execute:
npx json-server db.json
The server start at:
http://localhost:3000
The student records can be accessed using
http://localhost:3000/students.

(2) Implementation of HTTP methods.

a. GET method: The GET method is used to retrieve student records from JSON server.

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

b. Post method: The post method is used to insert a new student record into the server.

Code:

```
fetch("http://localhost:3000/students",{
method: "PUT",
headers: {
"Content-TYPE": "application/json"
},
body: JSON.stringify({
    name: "Chandu",
    course: "AIML",
    marks: 95
})
})
.then(response => response.json())
.then(data => {
    console.log(data);
})
.catch(error => {
    console.log(error);
});
```

output:

D- patch method: the patch method is used to update only selected fields of an existing student record.

Code:
```
fetch("http://localhost:3000/students/2", {
  method: "PATCH",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    marks: 96
  })
})
.then(response => response.json())
.then(data => {
  console.log(data);
})
.catch(error => {
  console.log(error);
});
```

output:

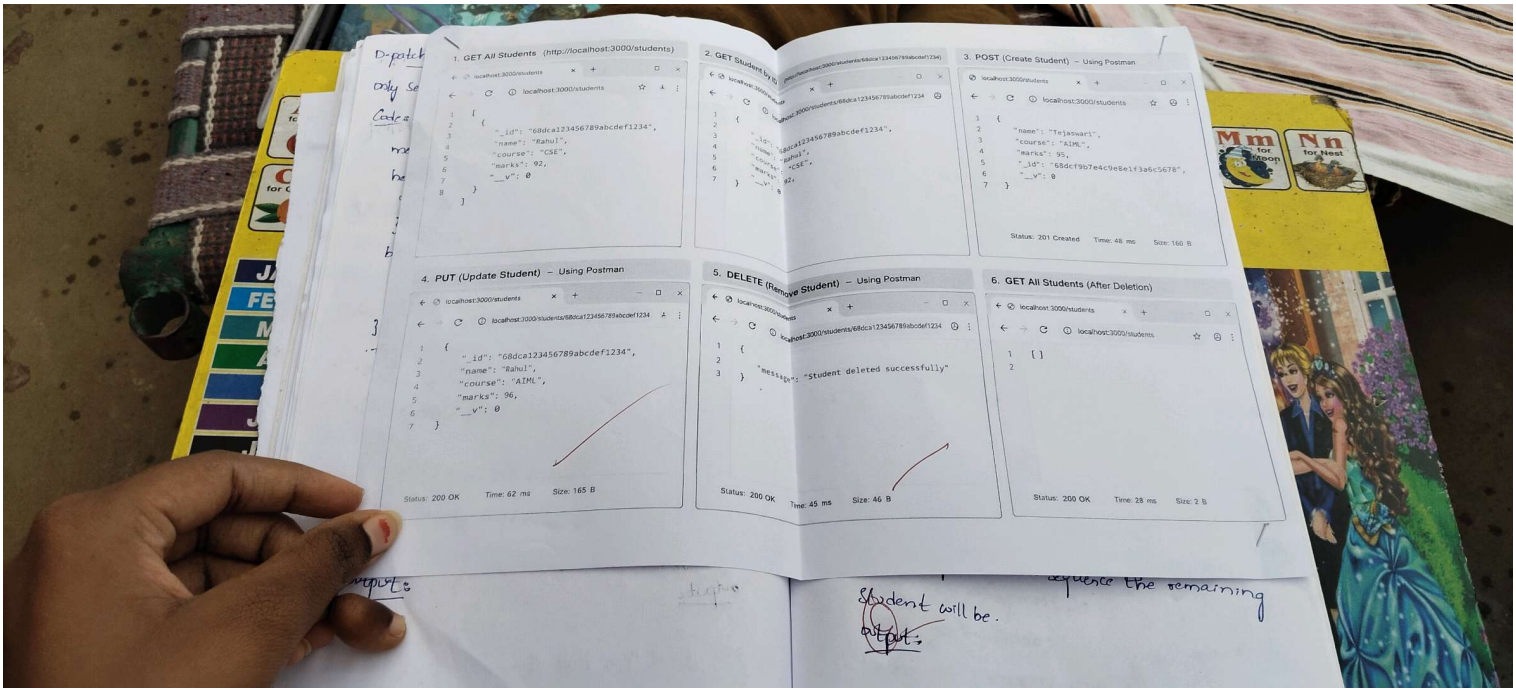
E- Delete method: The delete method is used to remove an existing student record from the server.

Code:
```
fetch("http://localhost:3000/students
  method: "DELETE",
})
.then(response => {
  if (response.ok) {
    console.log("student deleted succes
  }
})
.catch(error => {
  console.log(error)
});
```

output:

- After executing the POST, PUT, PATCH a DELETE operations in sequence the remaining student will be.

output:


1. GET All Students (http://localhost:3000/students)
2. GET Students by ID
3. POST (Create Student) – Using Postman
4. PUT (Update Student) – Using Postman
5. DELETE (Remove Student) – Using Postman
6. GET All Students (After Deletion)
Status: 200 OK
Time: 62 ms
Size: 165 B
Status: 200 OK
Time: 45 ms
Size: 46 B
Status: 200 OK
Time: 28 ms
Size: 2 B
Status: 201 Created
Time: 48 ms
Size: 160 B
Student will be.
output:

this, HTTP methods: GET, POST, PUT, PATCH, and DELETE were successfully implemented using JSON server. The CRUD operations were performed on student records. The data was retrieved, inserted, updated and deleted using HTTP requests.

Experiment-09

1) Develop CRUD operations using MongoDB.

Theory: MongoDB is a Nosql database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in database.

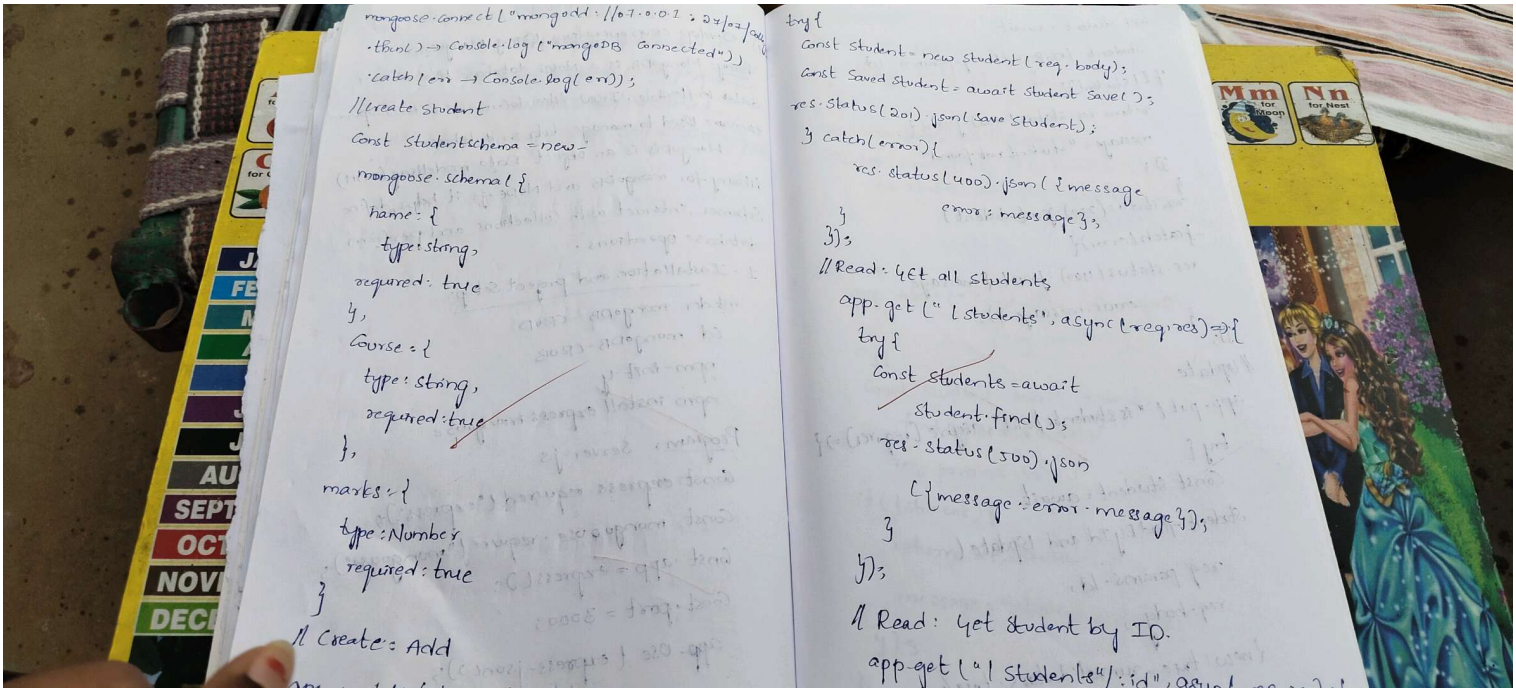
MongoDB is an object Data modelling (ODM) library for mongoDB and Node.js it helps define Schemas, interact with collections and perform database operations.

1. Installation and project setup:

```
mkdir mongoDB-CRUD
cd mongoDB-CRUD
npm init -y
npm install express mongoose
```

Program: Server.js

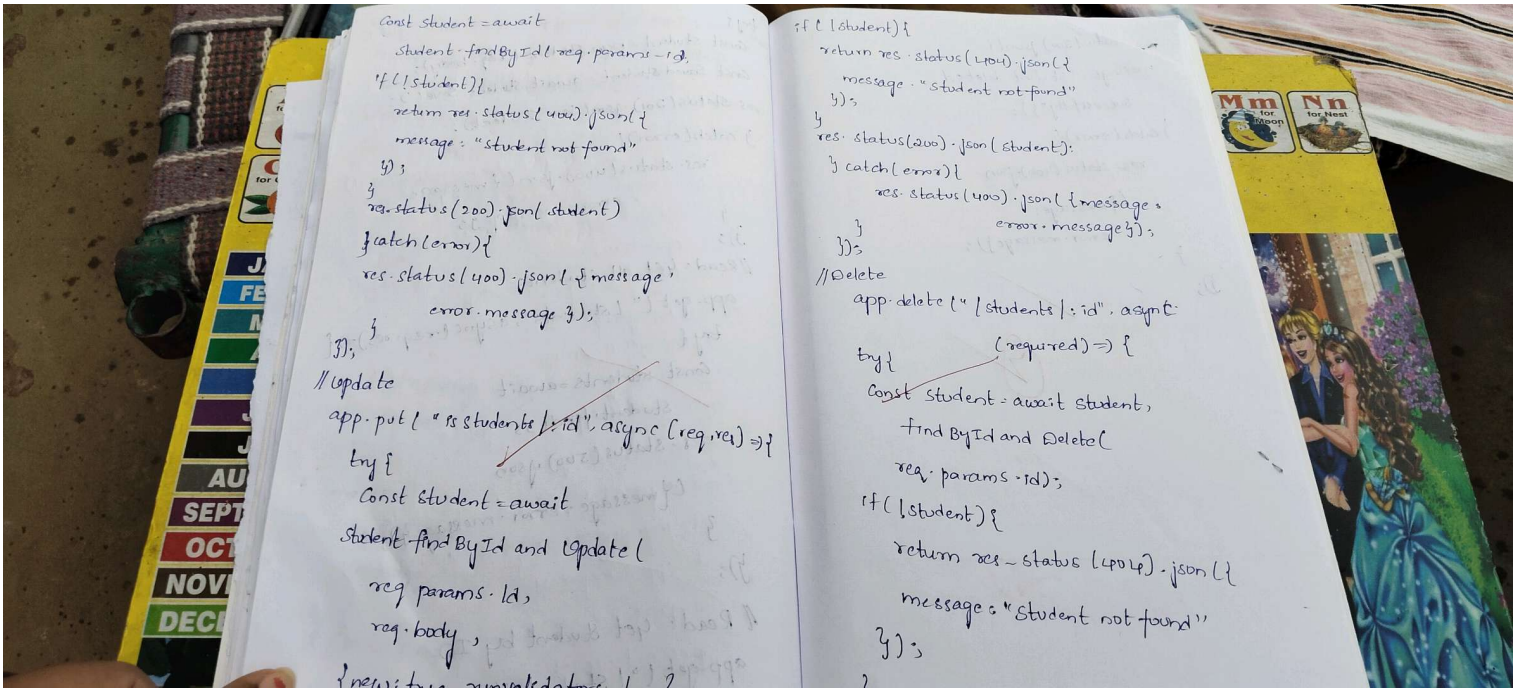
```
Const express required ("express");
Const mongoose.require ("mongoose");
Const app = express();
Const port = 3000;
app.Use (express.json());
```



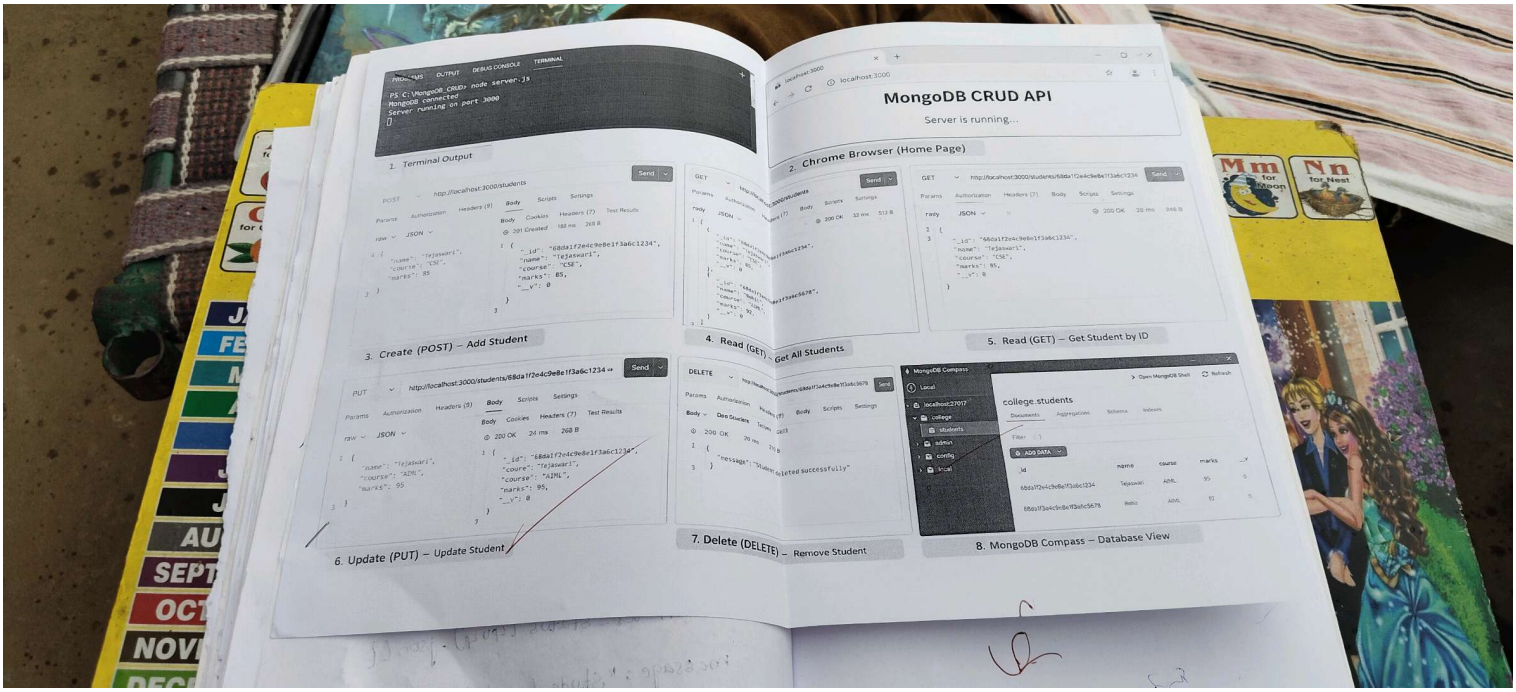
```
mongoose.connect("mongodb://127.0.0.1:27017/...
.then(() => console.log("mongoDB connected"))
.catch(err => console.log(err));

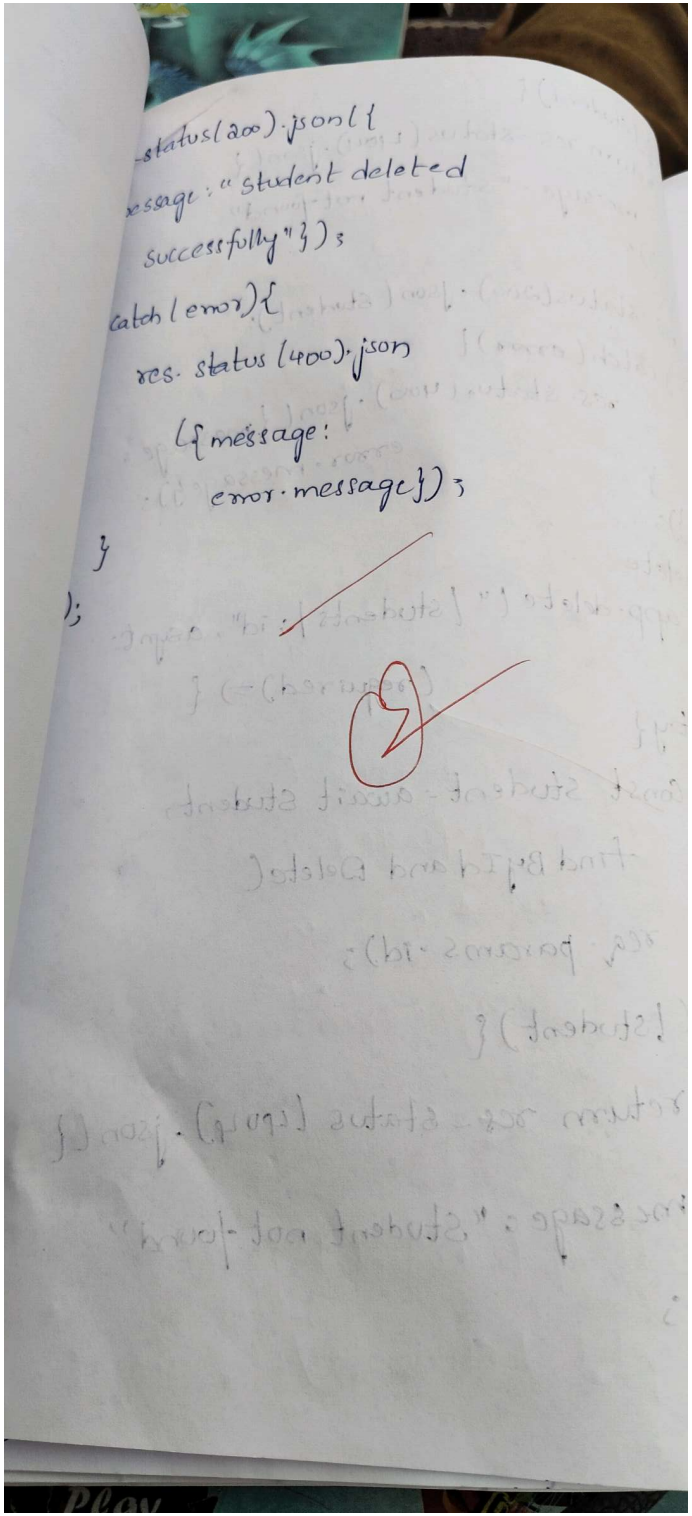
//Create Student
const StudentSchema = new
mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  Course: {
    type: String,
    required: true
  },
  marks: {
    type: Number,
    required: true
  }
// Create: Add
```

```
try{
const student = new Student(req.body);
const Saved Student = await Student.save();
res.status(201).json(Save Student);
} catch(error){
    res.status(400).json({message:
        error: message});
  }
});
// Read: Get all students
app.get("/students", async(req,res)=>{
  try{
    const students = await
        Student.find();
    res.status(500).json
      ({message: error.message});
    }
});
// Read: Get student by ID.
app.get("/students/:id", async
```



```
Const student = await
  student.findById(req.params.id);
if(!student){
  return res.status(404).json({
  message: "student not found"
  });
}
res.status(200).json(student)
}catch(error){
res.status(400).json({message:
      error.message});
  }
});
//Update
app.put("/:id", async(req,res)=>{
  try{
  Const student = await
  Student.findById and Update(
    req.params.id,
    req.body,
if(!student){
  return res.status(404).json({
    message: "student not found"
  });
}
res.status(200).json(student);
} catch(error){
    res.status(400).json({message:
                error.message});
  }
});
//Delete
app.delete("/students/:id", async
                (required) => {
try{
Const student = await Student.
    findById and Delete(
    req.params.id);
if(!student){
  return res.status(404).json({
  message: "Student not found"
});
```


1. Terminal Output
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
2. Chrome Browser (Home Page)
MongoDB CRUD API
Server is running...
3. Create (POST) – Add Student
4. Read (GET) – Get All Students
5. Read (GET) – Get Student by ID
6. Update (PUT) – Update Student
7. Delete (DELETE) – Remove Student
8. MongoDB Compass – Database View
college.students



```
-status(200).json({
essage: "student deleted
 successfully"});
catch(error){
 res.status(400).json
  ({message:
   error.message});
}
);
```